

```

import random

# Simulating network nodes with their clock times (in seconds)
class Node:
    def __init__(self, node_id, clock_time):
        self.node_id = node_id
        self.clock_time = clock_time

    def get_time(self):
        return self.clock_time

    def adjust_time(self, offset):
        self.clock_time += offset

    def __repr__(self):
        return f"Node {self.node_id}: {self.clock_time:.2f} sec"

class Coordinator(Node):
    def synchronize_clocks(self, nodes):
        print("Polling times from all nodes...")
        times = {}

        # Step 1: Collect time differences
        for node in nodes:
            if node.node_id == self.node_id:
                continue
            # Simulate network delay
            delay = random.uniform(0.1, 0.5)
            node_time = node.get_time() + delay / 2 # assume symmetric delay
            times[node.node_id] = node_time - self.clock_time
            print(f"Node {node.node_id} time: {node.get_time():.2f}, adjusted for
delay: {node_time:.2f}")

        # Add coordinator itself
        times[self.node_id] = 0.0

        # Step 2: Compute average offset (excluding outliers if needed)
        average_offset = sum(times.values()) / len(times)
        print(f"\nCalculated average offset: {average_offset:.2f} seconds")

        # Step 3: Send adjustment to each node
        for node in nodes:
            offset = average_offset - times[node.node_id]
            node.adjust_time(offset)
            print(f"Adjusting Node {node.node_id} by {offset:+.2f} seconds")

        print("\nSynchronization complete.\n")

```

```

# --- Example usage ---
if __name__ == "__main__":
    # Create nodes with different clock times
    nodes = [
        Coordinator(0, 100.0),    # Coordinator
        Node(1, 102.0),
        Node(2, 98.5),
        Node(3, 101.2),
        Node(4, 99.8),
    ]

    print("Initial times:")
    for node in nodes:
        print(node)

    coordinator = nodes[0]
    coordinator.synchronize_clocks(nodes)

    print("Final times after synchronization:")
    for node in nodes:
        print(node)

```

```

import socket
import threading
import json
import time

client_offsets = {}
client_connections = {}

def handle_client(conn, addr, node_id):
    global client_offsets
    print(f"Connected to node {node_id} at {addr}")

    while True:
        try:
            # Step 1: Ask for time
            conn.sendall(b'GET_TIME')
            data = conn.recv(1024).decode()
            if not data:
                break
            client_time = float(data)
            print(f"Node {node_id} time: {client_time}")
            client_offsets[node_id] = client_time
            break
        except:
            break

```

```

def start_server(host='localhost', port=8000, expected_clients=3):
    global client_connections
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((host, port))
    server.listen()

    print(f"Coordinator listening on {host}:{port}")

    node_id = 1
    while len(client_connections) < expected_clients:
        conn, addr = server.accept()
        client_connections[node_id] = conn
        threading.Thread(target=handle_client, args=(conn, addr, node_id)).start()
        node_id += 1

    # Wait for all clients to send times
    while len(client_offsets) < expected_clients:
        time.sleep(1)

    # Step 2: Compute average offset
    local_time = time.time()
    all_times = list(client_offsets.values()) + [local_time]
    average_time = sum(all_times) / len(all_times)
    print(f"Local time: {local_time}")
    print(f"Average network time: {average_time}")

    # Step 3: Send offset to each node
    for node_id, conn in client_connections.items():
        offset = average_time - client_offsets[node_id]
        conn.sendall(f"OFFSET:{offset}".encode())
        print(f"Sent offset {offset:+.2f} to node {node_id}")

    server.close()

```

```

import socket
import time
import random

```

```

def start_node(node_id, host='localhost', port=8000):
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((host, port))

    # Simulated clock with random drift
    drift = random.uniform(-5, 5)
    local_time = time.time() + drift

```

```

while True:
    data = client.recv(1024).decode()
    if data == 'GET_TIME':
        # Respond with current (drifted) time
        client.sendall(str(local_time).encode())
    elif data.startswith('OFFSET'):
        # Apply offset to adjust clock
        offset = float(data.split(':')[1])
        local_time += offset
        print(f"[Node {node_id}] Adjusted clock by {offset:+.2f} seconds")
        print(f"[Node {node_id}] New time: {local_time}")
        break

client.close()

if __name__ == "__main__":
    import sys
    node_id = int(sys.argv[1]) if len(sys.argv) > 1 else 1
    start_node(node_id)

```
